

Desenvolvimento para Dispositivos Móveis

Bibliotecas Dio, Provider e
Floor

Universidade Federal do Rio Grande do Norte

Pacotes para chamadas de API

- http e Dio são pacotes populares para realizar requisições HTTP em Flutter.
- Ambos os pacotes permitem interações com APIs REST, como envio de requisições GET, POST, PUT, DELETE, entre outras.
- **Diferença principal:** o Dio oferece mais funcionalidades integradas, como **interceptores**, **cancelamento** de requisições, e **suporte a uploads/downloads** de arquivos.

Pacote http

- O http é uma solução simples e fácil de usar para requisições básicas.
- Vantagens:
 - Simplicidade.
 - Leve e eficiente para tarefas pequenas.
- Limitações:
 - Não possui suporte nativo a interceptores ou cancelamento de requisições.
 - Não trata uploads e downloads de arquivos de forma otimizada.
 - Recursos avançados como retry, timeout customizado e manipulação mais avançada de cookies/cache precisam de mais configuração manual.

Pacote http

```
import 'package:http/http.dart' as http;

Future<void> fetchData() async {
  final response = await http.get(Uri.parse('https://api.example.com/data'));

  if (response.statusCode == 200) {
    print('Response: ${response.body}');
  } else {
    throw Exception('Failed to load data');
  }
}
```

Pacote Dio

- O Dio é um pacote mais completo e flexível para manipular requisições HTTP.
- Principais recursos:
 - **Interceptors:** Captura, modifica ou rejeita requisições e respostas antes de serem enviadas ou processadas.
 - **Retry:** Permite configurar tentativas automáticas em caso de falha.
 - **Timeout customizado:** Configura diferentes tempos de timeout para conexão, leitura e escrita.
 - **Suporte a multipart/form-data:** Para uploads de arquivos com facilidade.
 - **CancelToken:** Permite o cancelamento de requisições em andamento.
 - **Upload/Download progress:** Monitora o progresso de uploads e downloads.

Pacote Dio

```
import 'package:dio/dio.dart';

Future<void> fetchData() async {
  Dio dio = Dio();

  try {
    final response = await dio.get('https://api.example.com/data');
    print('Response: ${response.data}');
  } catch (e) {
    print('Failed to load data: $e');
  }
}
```

Comparação http e dio

Característica	http	Dio
Simplicidade	Alta	Média
Interceptors	Não	Sim
Cancelamento de requisições	Não	Sim (via <code>CancelToken</code>)
Suporte multipart	Manual	Sim (nativo)
Suporte a progress listeners	Não	Sim
Timeout customizado	Manual (com timer)	Sim (nativo)
Retry automático	Não	Sim (com <code>interceptor</code>)
Downloads em background	Limitado	Sim

Vantagens do Dio sobre o http

- **Interceptores:** Customização de requisições e respostas antes de serem enviadas ou processadas.
- **Cancelamento de requisições:** Evita requisições pendentes ou longas quando não são mais necessárias.
- **Suporte a uploads e downloads:** Gerenciamento eficiente de grandes quantidades de dados com barra de progresso.
- **Facilidade de configuração de timeout:** Diferentes tempos de timeout para conexão, leitura e escrita de dados.
- **Retry:** Tentativas automáticas em caso de falhas temporárias.

Exemplo

```
Dio dio = Dio();

dio.interceptors.add(InterceptorsWrapper(
  onRequest: (options, handler) {
    // Modifica a requisição antes de ser enviada
    print('Requisição sendo enviada: ${options.uri}');
    return handler.next(options); // continua com a requisição
  },
  onResponse: (response, handler) {
    // Manipula a resposta antes de ser passada para o chamador
    print('Resposta recebida: ${response.data}');
    return handler.next(response); // continua com a resposta
  },
  onError: (DioError e, handler) {
    // Trata erros de requisições
    print('Erro durante requisição: ${e.message}');
    return handler.next(e); // continua com o erro
  },
)
```

Exemplo

```
class ApiClient {  
    late final Dio _dio;  
  
    ApiClient({required String baseUrl}) {  
        _dio = Dio()  
        ..options.baseUrl = baseUrl  
        ..options.headers  
        ..interceptors.add(  
            LogInterceptor(  
                requestBody: true,  
                responseBody: true,  
            ),  
        );  
    }  
}
```

Exemplo

```
Future<List<MovieEntity>> getMovies({int? page, int? limit}) async {  
  final response = await _dio.get(  
    "/movies",  
    queryParameters: {  
      '_page': page,  
      '_per_page': limit,  
    },  
  );  
  if (response.statusCode != null && response.statusCode! >= 400) {  
    throw NetworkException(  
      statusCode: response.statusCode!,  
      message: response.statusMessage,  
    );  
  } else if (response.statusCode != null) {  
    final HttpPagedResult receivedData = HttpPagedResult.fromJson(response.data as Map<String, dynamic>);  
  
    return receivedData.data;  
  } else {  
    throw Exception('Unknown error');  
  }  
}
```

Introdução ao Provider

- Provider é uma biblioteca popular no Flutter para **gerenciamento de estado**, oferecendo uma maneira simples e eficiente de **compartilhar dados entre diferentes widgets**.
- Ele é baseado no princípio de **Inversão de Controle e Injeção de Dependências**, facilitando o gerenciamento e acesso ao estado da aplicação.
- Fornece uma abordagem **mais escalável** em comparação ao uso direto do setState, especialmente em aplicativos grandes e complexos.

Provider

- **Gerenciamento eficiente de estado:** Ideal para compartilhar informações entre widgets sem a necessidade de propagar dados manualmente com construtores.
- **Simplicidade:** Fornece uma interface de fácil uso para reagir a mudanças de estado.
- **Reatividade:** Atualiza a UI automaticamente quando o estado compartilhado é alterado.
- **Escalabilidade:** Funciona bem tanto em pequenas quanto em grandes aplicações.

Exemplo básico sem Provider

```
class Counter extends StatefulWidget {  
  @override  
  _CounterState createState() => _CounterState();  
}  
  
class _CounterState extends State<Counter> {  
  int count = 0;  
  
  void increment() {  
    setState(() {  
      count++;  
    });  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return Column(  
      children: [  
        Text('Contador: $count'),  
        ElevatedButton(onPressed: increment, child: Text('Incrementar'))  
      ],  
    );  
  }  
}
```

Exemplo básico com Provider

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => CounterModel(),
      child: MyApp(),
    ),
  );
}

class CounterModel extends ChangeNotifier {
  int _count = 0;

  int get count => _count;

  void increment() {
    _count++;
    notifyListeners();
  }
}
```

```
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('Provider Example')),
        body: Center(
          child: Consumer<CounterModel>(
            builder: (context, counter, child) =>
              Text('Contador: ${counter.count}'),
          ),
        ),
        floatingActionButton: FloatingActionButton(
          onPressed: () =>
            context.read<CounterModel>().increment(),
          child: Icon(Icons.add),
        ),
      ),
    );
  }
}
```

Exemplo básico com Provider

```
import 'package:flutter/material.dart';  
import 'package:provider/provider.dart';
```

```
void main() {  
  runApp(  
    ChangeNotifierProvider(  
      create: (_) => CounterModel(),  
      child: MyApp(),  
    ),  
  );  
}
```

```
class CounterModel extends ChangeNotifier {  
  int _count = 0;  
  
  int get count => _count;  
  
  void increment() {  
    _count++;  
    notifyListeners();  
  }  
}
```

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  
        appBar: AppBar(title: Text('Provider Example')),  
        body: Center(  
          child: Consumer<CounterModel>(  
            builder: (context, counter, child) =>  
              Text('Contador: ${counter.count}'),  
          ),  
        ),  
        floatingActionButton: FloatingActionButton(  
          onPressed: () =>  
            context.read<CounterModel>().increment(),  
          child: Icon(Icons.add),  
        ),  
      ),  
    );  
  }  
}
```

O widget **Consumer** é uma maneira conveniente de acessar e reagir a mudanças no estado gerenciado pelo **Provider**.

Exemplo básico com Provider

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
```

```
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => CounterModel(),
      child: MyApp(),
    ),
  );
}

class CounterModel extends ChangeNotifier {
  int _count = 0;

  int get count => _count;

  void increment() {
    _count++;
    notifyListeners();
  }
}
```

```
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('Provider Example')),
        body: Center(
          child: Consumer<CounterModel>(
            builder: (context, counter, child) =>
              Text('Contador: ${counter.count}'),
          ),
        ),
        floatingActionButton: FloatingActionButton(
          onPressed: () =>
            context.read<CounterModel>().increment(),
          child: Icon(Icons.add),
        ),
      ),
    );
  }
}
```

Consumer reconstrói automaticamente quando o valor count no CounterModel é alterado.

Vantagens do Provider

- **Desacoplamento:** Separação clara entre a lógica de negócios (modelo) e a interface de usuário.
- **Reatividade automática:** Atualiza a UI automaticamente sem necessidade de chamadas manuais para setState.
- **Escalabilidade:** O estado pode ser compartilhado em várias partes da aplicação sem a complexidade de passar dados manualmente entre widgets.
- **Modularidade:** Facilita a divisão do código em componentes menores e mais fáceis de testar.

Provider com múltiplos estados

- Com o **MultiProvider**, podemos fornecer diferentes tipos de estado a várias partes do aplicativo.
- Com o **MultiProvider**, widgets diferentes podem consumir estados diferentes sem conflito.

Provider com múltiplos estados

```
class UserModel extends ChangeNotifier {  
    String _name = 'Usuário';  
  
    String get name => _name;  
  
    void changeName(String newName) {  
        _name = newName;  
        notifyListeners();  
    }  
}  
  
class CounterModel extends ChangeNotifier {  
    int _count = 0;  
  
    int get count => _count;  
  
    void increment() {  
        _count++;  
        notifyListeners();  
    }  
}
```

Provider com múltiplos estados

```
void main() {  
  runApp(  
    MultiProvider(  
      providers: [  
        ChangeNotifierProvider(create: (_) => UserModel()),  
        ChangeNotifierProvider(create: (_) => CounterModel()),  
      ],  
      child: MyApp(),  
    ),  
  );  
}
```

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  
        appBar: AppBar(title: Text('Exemplo MultiProvider')),  
        body: Column(  
          mainAxisAlignment: MainAxisAlignment.center,  
          children: [  
            Consumer<UserModel>(  
              builder: (context, user, child) => Text('Usuário: ${user.name}'),  
            ),  
            Consumer<CounterModel>(  
              builder: (context, counter, child) => Text('Contador: ${counter.count}'),  
            ),  
          ],  
        ),  
        floatingActionButton: Column(  
          mainAxisAlignment: MainAxisAlignment.end,  
          children: [  
            FloatingActionButton(  
              onPressed: () => context.read<UserModel>().changeName('Novo Nome'),  
              child: Icon(Icons.person),  
            ),  
            SizedBox(height: 10),  
            FloatingActionButton(  
              onPressed: () => context.read<CounterModel>().increment(),  
              child: Icon(Icons.add),  
            ),  
          ],  
        ),  
      ),  
    ),  
  ),  
);
```

SingleChildWidget

- O **SingleChildWidget** é uma interface importante dentro do pacote **Provider** no Flutter. Ele é utilizado para widgets que encapsulam apenas um único filho, sendo particularmente útil em widgets que precisam ser usados dentro de uma **árvore de providers**, como o **ChangeNotifierProvider**, **Consumer**, e **StreamProvider**.
- No contexto do **Provider**, o **SingleChildWidget** é um contrato que diz que um widget pode ter um único widget filho. A maioria dos providers no pacote, como o **ChangeNotifierProvider** e o **FutureProvider**, implementa essa interface. Isso significa que esses providers encapsulam um único widget na árvore de widgets do Flutter.

SingleChildWidget

- A interface **SingleChildWidget** é particularmente útil quando você está trabalhando com múltiplos providers, especialmente com o **MultiProvider**. Ele ajuda a garantir que cada widget que está sendo combinado tenha uma estrutura consistente.
- **MultiProvider**: Quando você usa **MultiProvider** para fornecer múltiplos providers em uma árvore de widgets, o **SingleChildWidget** é fundamental. Ele garante que cada provider encapsule corretamente seu filho, respeitando a estrutura hierárquica.

SingleChildWidget exemplo

```
class UserModel extends ChangeNotifier {  
    String _name = 'Usuário';  
  
    String get name => _name;  
  
    void changeName(String newName) {  
        _name = newName;  
        notifyListeners();  
    }  
}  
  
class CounterModel extends ChangeNotifier {  
    int _count = 0;  
  
    int get count => _count;  
  
    void increment() {  
        _count++;  
        notifyListeners();  
    }  
}
```

SingleChildWidget exemplo

```
class ConfigureProviders {  
    final List<SingleChildWidget> providers;  
  
    ConfigureProviders({required this.providers});  
  
    static Future<ConfigureProviders> createDependencyTree() async {  
  
        final counterModel = CounterModel();  
        final userModel = UserModel();  
  
        return ConfigureProviders(providers: [  
            ChangeNotifierProvider<CounterModel>.value(value: counterModel),  
            ChangeNotifierProvider<UserModel>.value(value: userModel),  
        ]);  
    }  
}
```

SingleChildWidget exemplo

```
void main() async {  
  
  WidgetsFlutterBinding.ensureInitialized();  
  
  final data = await ConfigureProviders.createDependencyTree();  
  
  runApp(  
    MultiProvider(  
      providers: data.providers,  
      child: MyApp(),  
    ),  
  );  
}
```

SingleChildWidget exemplo

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
  
    UserModel userModel = Provider.of<UserModel>(context, listen: true);  
    CounterModel counterModel = Provider.of<CounterModel>(context, listen: true);  
  
    return MaterialApp(  
      home: Scaffold(  
        appBar: AppBar(title: Text('Exemplo MultiProvider')),  
        body: Column(  
          mainAxisAlignment: MainAxisAlignment.center,  
          children: [  
            Text('Usuário: ${userModel.name}'),  
            Text('Contador: ${counterModel.count}'),  
          ],  
        ),  
        floatingActionButton: Column(  
          mainAxisAlignment: MainAxisAlignment.end,  
          children: [  
            FloatingActionButton(  
              onPressed: () => userModel.changeName('Novo Nome'),  
              child: Icon(Icons.person),  
            ),  
            SizedBox(height: 10),  
            FloatingActionButton(  
              onPressed: () => counterModel.increment(),  
              child: Icon(Icons.add),  
            ),  
          ],  
        ),  
      ),  
    );  
  }  
}
```

Tipos de Provider

- No Flutter, o pacote **Provider** oferece diferentes tipos de providers para gerenciar o estado de forma eficiente e reativa. Cada tipo de provider serve para casos de uso específicos, permitindo a injeção de dependências e atualizações automáticas da interface de usuário quando o estado muda.

Provider

- O **Provider** é a implementação mais básica do pacote. Ele simplesmente fornece uma instância de um objeto para toda a árvore de widgets ou parte dela, mas não reativa ou notifica automaticamente os widgets sobre mudanças. Esse provider é usado quando você não precisa que a interface seja atualizada automaticamente.
- **Quando usar:** Para objetos que não precisam notificar mudanças, como classes que não estendem **Listenable**, **Stream**, ou **ChangeNotifier**.

ChangeNotifierProvider

- O **ChangeNotifierProvider** é uma das implementações mais utilizadas. Ele é projetado para fornecer uma instância de uma classe que estende `ChangeNotifier`. Essa classe possui o método `notifyListeners()`, que é chamado para notificar a UI sobre mudanças. O `ChangeNotifierProvider` automaticamente se reconstrói quando o `ChangeNotifier` associado invoca `notifyListeners()`.
- **Quando usar:** Quando você precisa de um modelo de estado reativo que avise a interface sobre mudanças, e esse estado é gerenciado por meio de `ChangeNotifier`.

ValueListenableProvider

- O **ValueListenableProvider** é usado para fornecer valores que são gerenciados por um objeto que implementa **ValueListenable**. Um **ValueListenable** é uma classe que emite valores e pode ser observada, e o mais comum é o uso de **ValueNotifier**.
- **Quando usar**: Quando você já está usando um **ValueListenable**, como um **ValueNotifier**, para monitorar mudanças no valor e quer que os widgets dependentes sejam atualizados automaticamente.

ValueListenableProvider

```
ValueListenableProvider<int>(  
    create: (context) => ValueNotifier<int>(0),  
    child: MyApp(),  
);  
  
ValueNotifier<int> counter = ValueNotifier(0);  
  
void incrementCounter() {  
    counter.value++;  
}
```

FutureProvider

- O **FutureProvider** é usado para fornecer o valor de um **Future** e atualizar a interface de usuário automaticamente quando o **Future** é resolvido. O valor é carregado de forma assíncrona e a UI é reconstruída quando o **Future** é completado.
- **Quando usar:** Quando você quer usar um valor que vem de um **Future** e a UI deve ser atualizada quando o futuro é resolvido.

StreamProvider

- O **StreamProvider** é usado para expor um **Stream** para a árvore de widgets e notificar automaticamente os widgets dependentes sempre que o stream emitir novos valores. O **StreamProvider** escuta o stream e reconstrói os widgets conforme novos dados são recebidos.
- **Quando usar:** Quando você está lidando com dados assíncronos que vêm de um **Stream** e quer que a interface de usuário seja atualizada automaticamente conforme o stream emite novos valores.

Pacote Floor

- O Floor é uma biblioteca de ORM (Object Relational Mapping) para Flutter e Dart que simplifica a interação com bancos de dados SQLite.
- Principais características:
 - Geração automática de código.
 - Suporte a migrações.
 - Facilita a persistência de dados no SQLite.
 - Baseado em anotação (annotation-based).
 - Integração fluida com Streams, permitindo reatividade.

Pacote Floor

- Para usar o Floor, você precisa adicionar as dependências no seu arquivo pubspec.yaml

```
dependencies:  
  flutter:  
    sdk: flutter  
  floor: ^1.0.0 # Verifique a versão atual  
  floor_generator: ^1.0.0 # Geração de código  
  sqflite: ^2.0.0 # Para suporte ao SQLite  
  
dev_dependencies:  
  build_runner: ^2.0.0 # Para gerar o código
```

- Para gerar os arquivos use o comando:

```
flutter pub run build_runner build
```

Funcionamento

- O **Floor** usa anotações para mapear classes Dart para tabelas do banco de dados SQLite.
- Componentes principais:
 - **Entity**: Representa uma tabela no banco de dados.
 - **DAO (Data Access Object)**: Define os métodos para acessar os dados.
 - **Database**: Abstração do banco de dados, que integra o DAO e gerencia as instâncias de conexão.

Entity

- Uma Entity representa uma tabela no banco de dados. Cada campo da classe corresponde a uma coluna da tabela.

```
import 'package:floor/floor.dart';

@Entity(tableName: 'users')
class User {
  @PrimaryKey(autoGenerate: true)
  final int? id;

  final String name;
  final int age;

  User({this.id, required this.name, required this.age});
}
```

Data Access Object

- O DAO é responsável por fornecer métodos para acessar os dados da entidade. Usamos a anotação `@dao` para definir a interface com as consultas SQL.

Data Access Object

```
@dao
abstract class UserDao {

    @Query('SELECT * FROM users')
    Future<List<User>> findAllUsers();

    @Query('SELECT * FROM users WHERE id = :id')
    Stream<User?> findUserId(int id);

    @insert
    Future<void> insertUser(User user);

    @delete
    Future<void> deleteUser(User user);

    @update
    Future<void> updateUser(User user);
}
```

Database

- Para criar o banco de dados, você define uma classe abstrata que estende FloorDatabase e inclui todos os DAOs que ele deve gerenciar.

Criação do banco de dados

```
@Database(version: 1, entities: [User])  
abstract class AppDatabase extends FloorDatabase {  
    UserDao get userDao;  
}
```

Uso do banco de dados

```
final database = await $FloorAppDatabase
    .databaseBuilder('app_database.db')
    .build();

final userDao = database.userDao;
```

Bibliografia

- <https://pub.dev/packages/provider/>
- <https://pinchbv.github.io/floor/getting-started/>
- <https://pub.dev/packages/dio>